

강화 학습 프로젝트

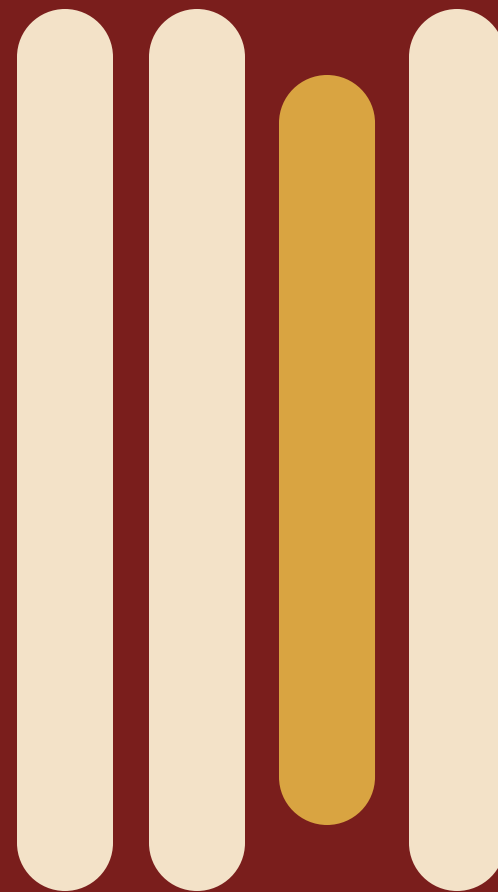
윗놀이 강화학습 프로젝트

윗놀이 게임에서의 강화학습 적용

A72046 남준우 / A72067 이기쁨

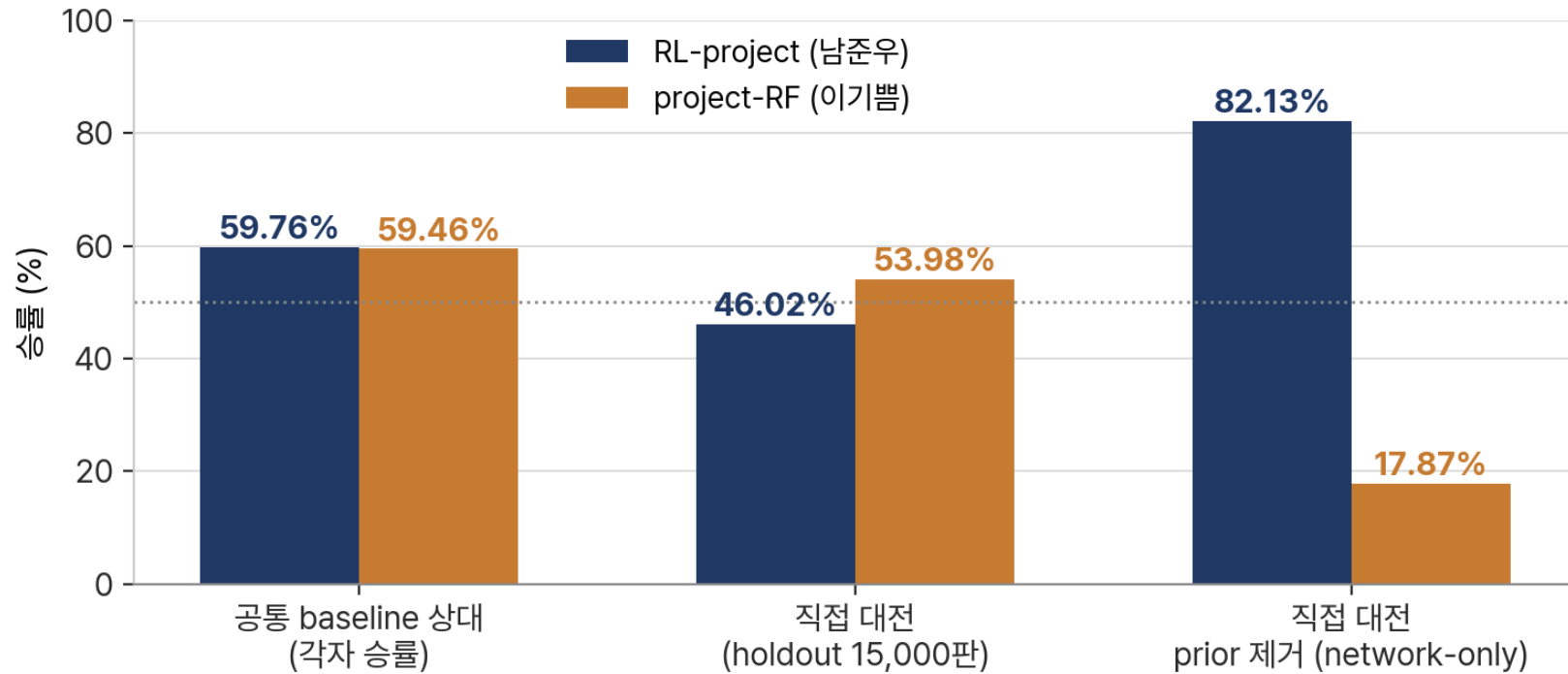
Github Link: <https://github.com/David-Nam/Yutnori-RL-Project>

2026. 06. 12



최종 결과 요약

두 Agent를 3가지 방식으로 평가하고, 평가 기준에 따라 순위가 달라지는 것을 확인



공통 baseline 상대로는 동급 (59.76% vs 59.46%) · 직접 대전은 project-RF 우세 (53.98%) · 신경망 단독은 RL-project 압도 (82.13%)

목차

01

배경 및 주제 기획

배경, 공통 평가 환경, 학습 데이터, Action, 평가방법

02

State vs Reward 설계 비교

project-RF: 전략 강화학습 · RL-project: 순수 강화학습 · 설계 비교표

03

비교 평가

실험 셋업, 공통 Rule-Based 상대 평가, Agent 직접 대결, 행동 분석 및 결과 해석

04

해석과 결론

프로젝트 결과 보고, 한계점, 핵심 결론, 프로젝트의 향후 방향

01

배경 및 주제 기획



배경 — 한국의 전통적인 윷놀이를 강화학습 할 수 있을까?

동일한 윷놀이 환경에서 서로 다른 강화학습 설계 방식 비교 / 객관적인 성능 검증을 위한 평가 프로토콜 설계

공통 규칙 기반 평가 환경 구축

- 동일한 윷놀이 규칙 적용
- 동일한 Action Space 정의
- 동일 Seed 및 선공·후공 조건 적용
- 5,000판 Paired Evaluation 수행

자유롭게 설계한 것

- observation과 reward
- 강화학습 알고리즘과 network 구조
- imitation · self-play · curriculum 사용 여부
- heuristic 결합 여부, 학습량과 hyperparameter

핵심 주제 서로 다른 설계로 강화학습을 학습하여 각각 윷놀이와의 대결에서 승률을 높일 수 있는지?

공통 평가 환경

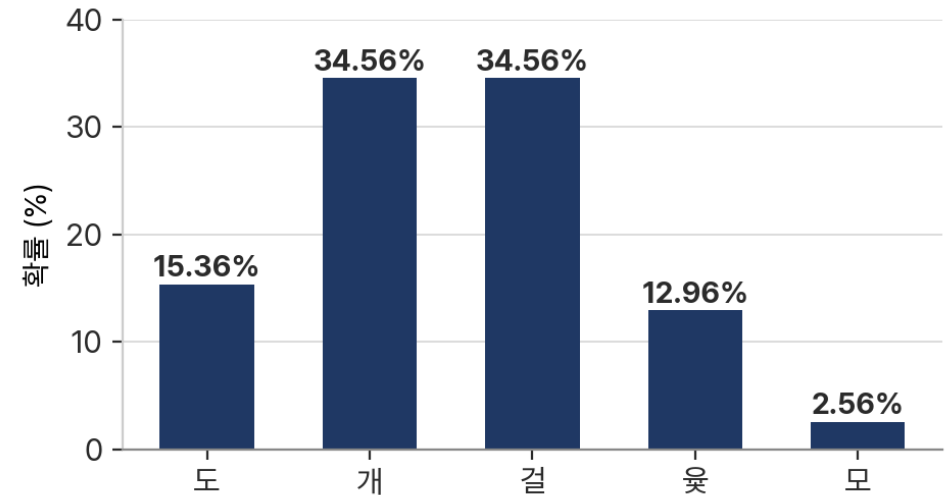
두 최종 모델이 모두 실행 가능한 표준 윗놀이 평가 환경

공통 게임 규칙

- 플레이어 2명, 플레이어당 말 4개
- 잡기(보너스 던지기 1회), 업기, 자동 지름길
- 윗·모 추가 던지기, HOME 통과 시 완주
- action 20개 = 말 4 × 윗 결과 5
- 뒷도 제외
- 모든 말 완주 시 종료

공통 평가 환경의 목적 -

알고리즘 성능만 비교하기 위해 환경 차이를 제거



공통 윗 결과 확률

학습 데이터 — Simulation Episode

별도의 외부 데이터셋 없이, 환경에서 직접 생성한 게임 기록으로 학습

데이터 생성 방식 (공통)

- self-play / opponent-play로 게임을 시뮬레이션하며 episode 생성
- episode = 윗놀이 한 판, 한쪽 말 4개가 모두 완주하면 종료
- state → action → reward → next state 전이를 그대로 학습에 사용
- 별도 preprocessing 없음 — 환경이 observation 인코딩까지 담당

프로젝트별 데이터 규모

- RL-project:
50M timesteps ≈ Seed당 약 152~154만 episode (3 seeds)
- project-RF:
Teacher 시연 8,000 샘플 (전술 상태 oversampling 적용)
+ mixed-opponent fine-tuning episode
- 데이터의 단위와 양 자체가 두 설계 철학의 차이를 반영

주의 학습 episode 통계(학습 중 승률 등)는 변화하는 정책의 평균이므로, 최종 deterministic 평가 수치와 직접 비교하지 않는다.

Action — 윷은 운, 선택은 전략

윷 던지기는 agent의 action이 아님

4

움직일 말 선택

×

5

사용할 윷 결과 선택

=

20

Action 공간

동작 방식과 두 프로젝트의 차이

- 던진 윷 결과(윷·모 보너스 포함)는 pool에 쌓이고, agent는 결과 하나를 골라 말 하나에 적용
- 불가능한 조합은 양쪽 모두 action mask로 차단 — 모든 평가에서 illegal action 0건
- encoding은 서로 다름: project-RF는 (윷결과 × 4 + 말), RL-project는 (말 × 5 + 윷결과)
- 직접 비교 시 state·action 변환 adapter로 연결 (A3·A4는 거리 보존 투영)

평가 방법— 세 가지 평가 방식

Paired Evaluation (모든 평가 공통)

- base seed마다 윷 시퀀스 고정 → 같은 seed에서 선공·후공을 한 번씩 교환 ($2,500 \text{ seed} \times 2 = 5,000$ 판)
- deterministic policy · 미래 윷 결과와 평가 RNG 참조 금지 · illegal action과 error 집계

공통 Rule-based 상대 평가

같은 고정 Rule-based Agent를
각자 얼마나 이기는가

두 Agent 직접 대전

두 최종 agent가 같은 윷 시퀀스에서
맞붙으면 누가 이기는가

Ablation — Prior 제거

project-RF의 tactical prior를 제거하면
결과가 어떻게 바뀌는가

내부 tournament 결과(각 프로젝트 안에서의 비교)는 상대 구성과 환경이 달라 프로젝트 간 우열 판단에는 사용하지 않는다.

공통 평가 Baseline — Rule-based Agent

행동 결정 방식

- 매 턴 모든 legal action의 점수를 계산해 최고점 선택
- 동점이면 가장 작은 action ID 선택 (tie-break 고정)
- 잡기·완주 같은 단기 전술에 매우 강한 기준선

통과 기준 — 왜 60%인가

- 윗놀이는 윗 던지기 운이 승패에 크게 개입
→ 실력이 같으면 승률은 50%에 수렴
- 운의 비중과 운영(전략)이 개입할 수 있는 폭을 고려해, 의미 있는 전략적 우위 수준을 60%로 설정
- 5,000판 기준 무작위 변동은 약 $\pm 1.4\%$ p
→ 60%는 우연으로 도달할 수 없는 수준 (3,000승 이상)

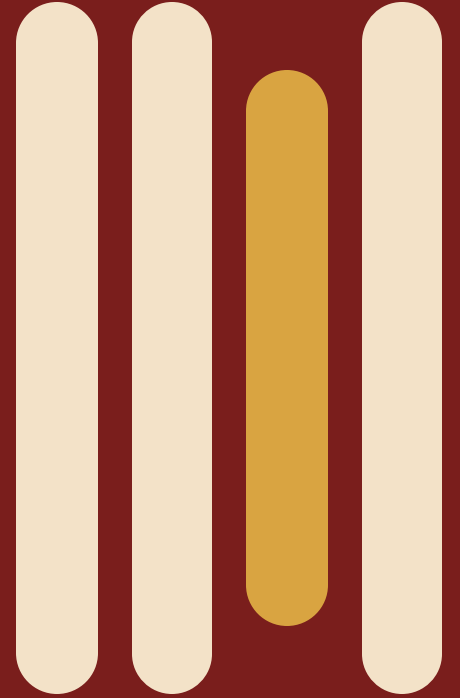
Reward Score

행동	점수
말 완주	+ 100
상대 말 잡기	+ 50
대기 중인 말 출발	+ 5
업힌 말 이동	+ 4 × (stack-1)
완주까지 거리	-0.5 × 거리

점수식은 같지만 동점 처리가 다른 옛 버전(큰 ID 선택)도 존재
작은 차이가 RL 성능에 큰 영향을 준다는 사실은 깨닫게 됨

02

강화학습 설계 비교



순수 강화학습과 전략 강화학습을 비교

두 설계 모두 잡기·완주·거리·위험이라는 같은 도메인 지식을 사용한다

project-RF — 전략 강화학습

전술 지식 (잡기·완주·위험·반격)



학습: StrategicValue teacher 모방 (KL distillation)



추론: PPO logits + $2.5 \times$ tactical 점수



최종 행동 = network + rule의 합작

분류: project-RF = RL + Imitation + Rule Prior Hybrid

RL-project — 순수 강화학습

전술 지식 (잡기·완주·위험·rf_score)



입력: action 별 tactical feature로 관측에 제공



학습: terminal 승패 보상만으로 40M 학습



최종 행동 = PPO network 단독 선택

RL-project = Pure PPO action selection + State Engineering

1# project-RF — 초기 설계 방향

이기쁨 — 252차원 engineered state + 강한 dense shaping

Dueling DQN

- Value Based RL 대표 알고리즘
- Q-value 학습으로 가치 함수 기반 학습이 가능한지 검증
- 일반 DQN 보다 상태 가치와 행동 가치를 분리하는 Dueling DQN 이
더 안정적으로 학습 할것이라 생각하여 선정
- 초기 가설
같은 상태에서 좋은 행동과 나쁜 행동의 차이가 크니 학습이
가능할 것
- 결과 : 장시간 GPU 학습에도 승률 약 20%

실험 결과 - Win rate 20%

원인	reason
Sparse Reward	승패가 게임 종료 시에만 결정
Long Horizon	50 턴 전 행동이 승패에 영향
Variable Action Space	가능한 행동의 수가 계속 변함
Opponent 존재	Non-stationary

프로젝트 초기에 Dueling DQN을 적용하여 윗놀이 전략 학습 가능성을 검증했으나 승률이 약 20% 수준에 머물렀다. 이에 따라 팀원의 순수 RL과 다른 State Engineering 과 Reward Shaping을 활용한 Hybrid PPO 구조를 설계하여 성능 향상을 시도함

1# project-RF — State와 Reward

이기쁨 — 252차원 engineered state + 강한 dense shaping

Engineered State (252차원)

- 양측 말 4개 위치 one-hot + pending 윷 결과
- 즉시 잡기.완주.지름길 가능 여부 플래그
- 현재 잡힐 위험 여부, 양측 완주 비율
- 양측 각 말의 완주까지 남은 거리
- Raw / Board / Engineered / RiskAware encoder를 구현해 비교
→ state 설계 자체가 핵심 실험 축

Capture-aware Dense Reward

사건	보상
승리 / 패배	+ 100 / -100
말 완주	+ 30
상대 말 잡기 / 잡힘	+ 35 / -35
잡을 수 있는데 안 잡음	-10
위험 상태 / 위험 탈출	-15 / + 5

전술 행동을 빠르게 유도하고 행동 근거를 설명할 수 있는 대신
가중치 튜닝 비용과 설계자 편향이 존재

1# project-RF — Teacher Imitation과 학습 상대

강한 teacher의 판단 분포를 PPO에 증류

StrategicValue Teacher

- value network + heuristic 점수 결합 agent (내부 최강)
- 완주 150 · 잡기 50 · 위험 34 · 상대 반격 20 가중치
- 정답 하나가 아니라 action별 점수 → softmax 분포로 변환
- PPO policy와의 KL divergence 최소화로 모방 학습

Mixed-opponent Fine-tuning

- stage 1: Random 50% + StrategicRule 50%
- stage 2: Random 20% + StrategicRule 40% + StrategicValue 40%
- 단일 상대 과적응을 줄이고 다양한 행동 분포에 대응

중요 상태 Oversampling

상태	가중치
잡기 가능	× 5
잡힐 위험	× 5
완주 가능	× 3
지름길 가능	× 2
일반 상태	× 1

승패를 가르는 전술 상태를 일반 상태보다 강하게 학습

1# project-RF — Inference-time Tactical Prior

최종 행동은 network 출력만으로 결정되지 않는다

$$\text{Final_logits} = \text{ppo_logits} + 2.5 \times \text{tactical_bonus}$$

Tactical bonus가 평가하는 것

- 즉시 승리·완주·잡기, 진행 거리, 지름길 진입
- 잡기 기회를 놓쳤는지, 이동 후 잡힐 위험
- 상대가 다음 행동으로 잡기·완주를 할 수 있는지
- 완주 직전 말을 두고 새 말을 출발시키는지

내부 개발 성과 (project-RF 내부 비교)

- 내부 7,000게임 tournament: Hybrid PPO 61.76%
- 잡기 기회 성공률 91.14%
- PPO Baseline 32% → imitation·tactical 설계로 약 2배 개선
- 단, 내부 결과는 상대·환경이 달라 프로젝트 간 비교에는 미사용

따라서 분류는 Pure PPO가 아니라 RL + Imitation + Rule Prior의 Hybrid PPO — 이 구조가 3부 결과 해석의 열쇠가 된다

1# project-RF — Hyperparameter와 후보 탐색

imitation 후보 4종을 비교해 최종 설정 선택

최종 PPO 설정 (자체 PyTorch 구현)

항목	값
learning rate	0.0001
gamma / GAE lambda	0.99 / 0.95
PPO clip	0.15
entropy coefficient	0.03
rollout / batch	1,024 / 128
network	256-256 ReLU

Imitation 후보 비교 (teacher 상대 평가)

후보	lr	gamma	GAE	승률
0 (선택)	1e-4	0.99	0.95	25.2%
1	5e-5	0.995	0.98	24.5%
2	1e-4	0.995	0.95	24.5%
3	5e-5	0.99	0.98	24.0%

승률은 최강 내부 agent인 StrategicValue teacher 상대 기준 — 절대값보다 후보 간 상대 비교에 사용

이후 단계 후보 0을 ppo_capture_imitation으로 선택 → mixed-opponent fine-tuning(ppo_tactical)은 마지막 stage에서 오히려 하락해 후보 0이 대표로 확정

2# RL-project — Tactical Observation과 Terminal Reward

남준우 — 행동의 의미를 관측으로 제공하고, 보상은 승패만

Action-level Tactical Observation (253차원)

- base 53차원 (말 위치·상태·stack·윳 pool)
- + 20개 action마다 10개 전술 feature = 200차원
- feature: legal, 잡기(수), 완주(수), 이동 말 수, 새 말 출발, stack 크기, 이동 후 거리, rf_score
 - rf_score = rule agent의 행동 점수
→ rule 지식이 입력으로 들어가지만 행동 강제는 없음

Terminal Reward

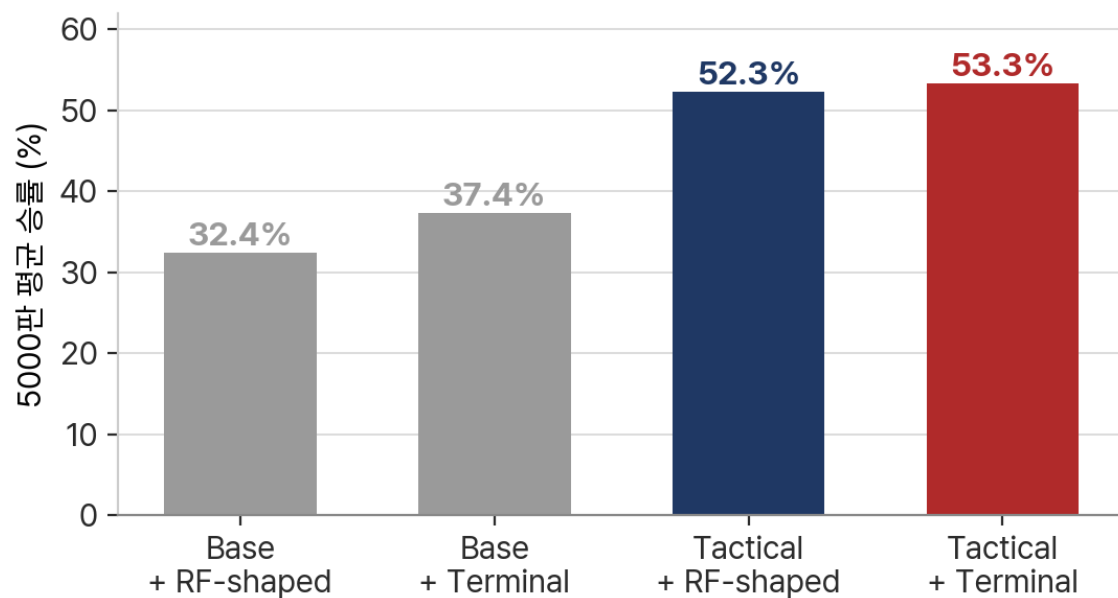
- 승리 + 1, 패배 -1, 그 외 0 (중간 보상 x)
- shaping 실험 결과 잡기 성향만 강해지고 승률은 오히려 하락
- “ 관측은 풍부하게, 보상은 단순하게” 가 설계 원칙

학습 구성

- sb3-contrib MaskablePPO
- policy/value 64-64 Tanh (작은 network)
- 학습 상대: common_rule_based 고정
- seed 0·1·2 × 40M timesteps
- SubprocVecEnv 12 env, A100
- 추론: deterministic policy 단독

2# RL-project — 성능 개선 실험

observation 2종 × reward 2종, 3-seed 후보 실험 (3M)

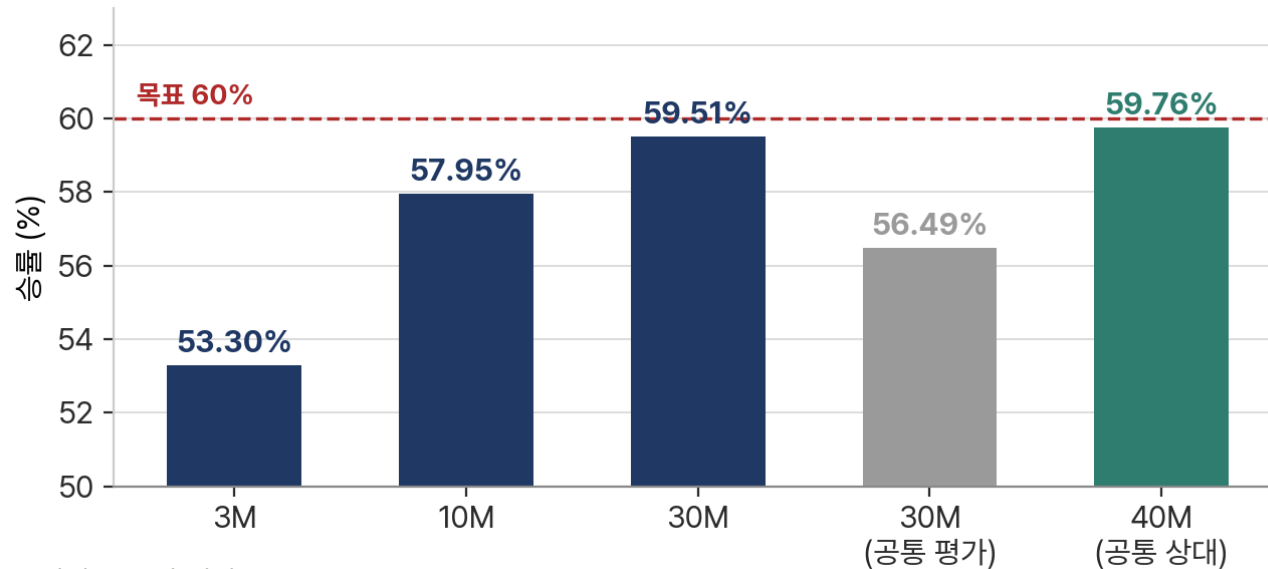


발견

- 최대 개선 요인은 tactical observation — base 대비 약 +15.9%p
- reward shaping은 오히려 역효과 (terminal이 우세)
- 결론: 이 게임에서는 reward 설계보다 state 표현이 지배적
- Tactical + Terminal 조합으로 장기 학습 확정

2# RL-project — 40M까지의 학습 과정

- 1 - 10M : tactical + terminal PPO를 10M 학습한 뒤, 각 모델을 공식 조건인 RF Agent 상대 5000판으로 평가
- 2 - 30M : 같은 PPO 후보를 seed별 fresh training하고, 12개 CPU core를 활용하도록 SubprocVecEnv와 n_envs=12를 사용했고, 학습은 A100 GPU에서 수행
- 3 - 40M : 공통 opponent를 직접 학습 상대로 사용해 같은 tactical + terminal PPO를 seed별 다시 학습



*파랑 = 초기 평가
*회색 = 공통 기준 재평가
*초록 = 공통 상대 재학습

중요한 교훈

- 평가 기준을 공통으로 통일하자 -3.02%p 하락
- 원인의 대부분은 상대 tie-break 변경 (-2.29%p)
- 학습 상대를 공통 agent로 교체 → +3.27%p 회복

최종 (40M × 3 seeds) 평균 59.76%, 최고 seed 1의 60.46% — seed 1·2가 60% 통과

2# RL-project — 알고리즘과 Hyperparameter

sb3-contrib MaskablePPO — 40M 장기 학습 설정

최종 PPO 설정 (legacy 40M)

항목	값
learning rate	0.0003
gamma / GAE lambda	0.99 / 0.95
n_steps / batch size	2,048 / 2,048
entropy coefficient	0
parallel env	SubprocVecEnv × 12
network	64-64 Tanh (policy/value)

실험 예산의 배분

- hyperparameter sweep 대신 observation × reward 조합 비교(4종 × 3 seed)에 실험 예산을 집중
- sb3 기본값 기반 설정을 고정하고 학습량·상대 정렬로 성능 확보

학습 안정장치

- MaskablePPO로 illegal action을 분포에서 제거
- opponent 선공 opening에서 terminal이 되는 reset 폐기 (경계 조건 버그 수정 후 회귀 테스트 추가)

두 모델의 Comparison Points

	project-RF (이기쁨)	RL-project (남준우)
모델 유형	RL + Imitation + Rule Prior Hybrid	Pure PPO + State Engineering
지식 주입 위치	정책(imitation) + 추론(prior)	관측(observation feature)
State	252차원 engineered state	253차원 action 별 tactical feature
Reward	강한 dense capture-aware shaping	terminal + 1 / -1
Network	256-256 ReLU (자체 PyTorch PPO)	64-64 Tanh (sb3 MaskablePPO)
학습 상대	teacher + mixed opponent	고정 common rule-based
학습 규모	imitation 중심 + fine-tuning, 단일 checkpoint	40M timesteps × 3 seeds
추론	PPO logits + 2.5 × tactical bonus	deterministic PPO 단독

03

비교 평가



실험 셋업

평가 지표는 통일하고, 학습 인프라는 각자의 설계에 맞게 사용

Evaluation Metric (모든 비교 공통)

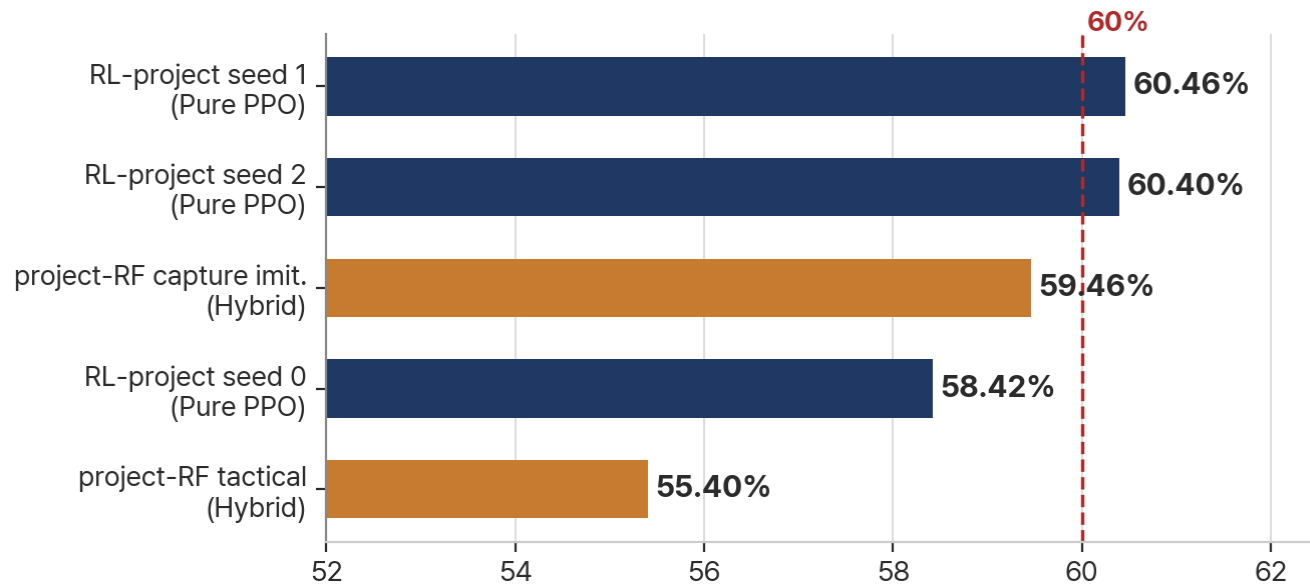
- paired 5,000판 승률 (base seed 2,500 × 선평·후공) · 95% Wilson 신뢰구간 · 선평/후공 분리 승률
- illegal action·evaluation error 집계 (전 평가 0건) · deterministic policy · 미래 정보 참조 금지

	RL-project	project-RF
학습 장치	NVIDIA A100 GPU	CPU 12 threads
학습 규모	40M timesteps × 3 seeds	imitation + fine-tuning, 단일 checkpoint
학습 시간	시드당 약 4시간 (12 env 병렬)	수 시간 내 (시연 수집 + distillation)
프레임워크	sb3-contrib MaskablePPO	자체 PyTorch PPO

학습 인프라와 학습 budget이 동일하지 않으므로, 본 비교는 sample efficiency 비교가 아니라 최종 산출된 agent 성능 비교로 해석해야 한다

공통 Rule-based 상대 평가

같은 고정 rule agent를 상대로 한 paired 5,000판 (모델·시드별)



파랑: RL-project (3 seeds) · 주황: project-RF (2 checkpoints)

0.30%p

3-seed 평균 59.76% vs 대표 59.46%

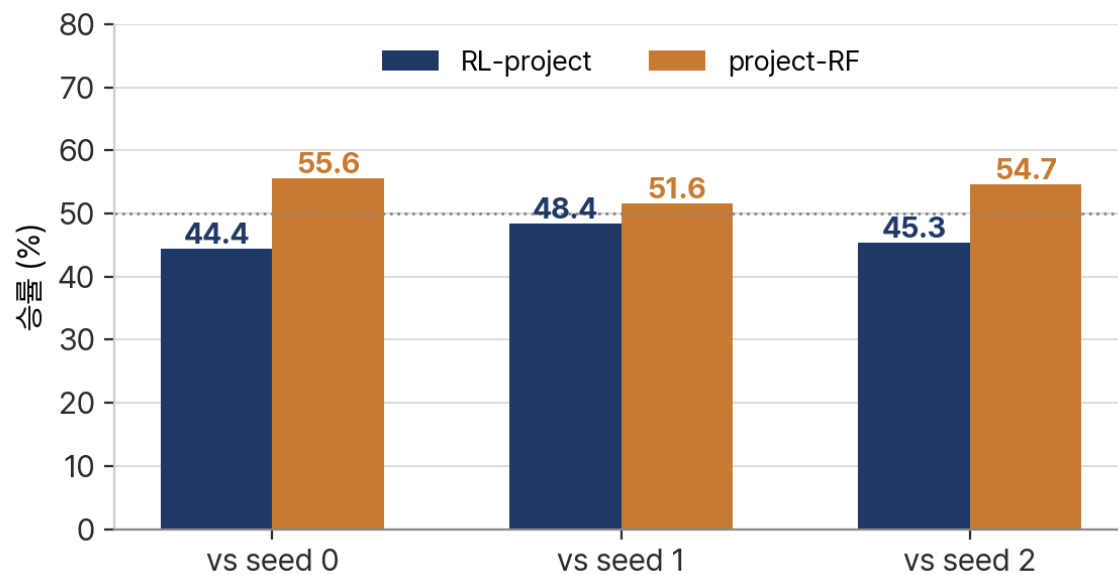
유사한 성능, 성능 차이는 크지 않음

읽는 법

- 두 설계 모두 baseline 상대 약 60% 수준 도달
- 60% 엄격 통과는 RL-project seed 1·2 (project-RF는 27승 부족)
- 단, seed 수와 학습 budget이 달라 우열로 확대 해석 금지

Final - 두 Agent 직접 대전

강화학습 vs 강화학습 — 같은 옷 시퀀스에서 선공·후공 교환



독립 holdout (seed 500000~502499) — RL-project 학습 seed별 5,000판

53.98%

project-RF Hybrid 합산 승률 (holdout 15,000판)

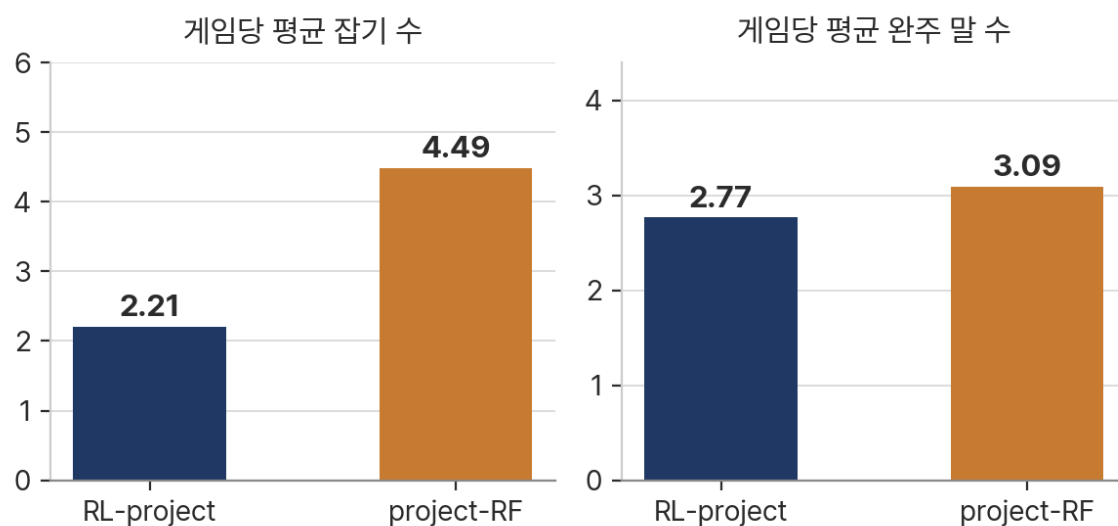
*95% Wilson CI 53.18~ 54.78%

결과의 견고함

- 기존 구간(100000~)에서도 52.92%로 같은 방향
- 두 구간 합산 30,000판: 53.45% — 우세 일관
- RL-project는 seed 1만 거의 대등 (48.36%)

직접 대전 행동 분석

head-to-head 5,000판의 게임 내 행동 통계



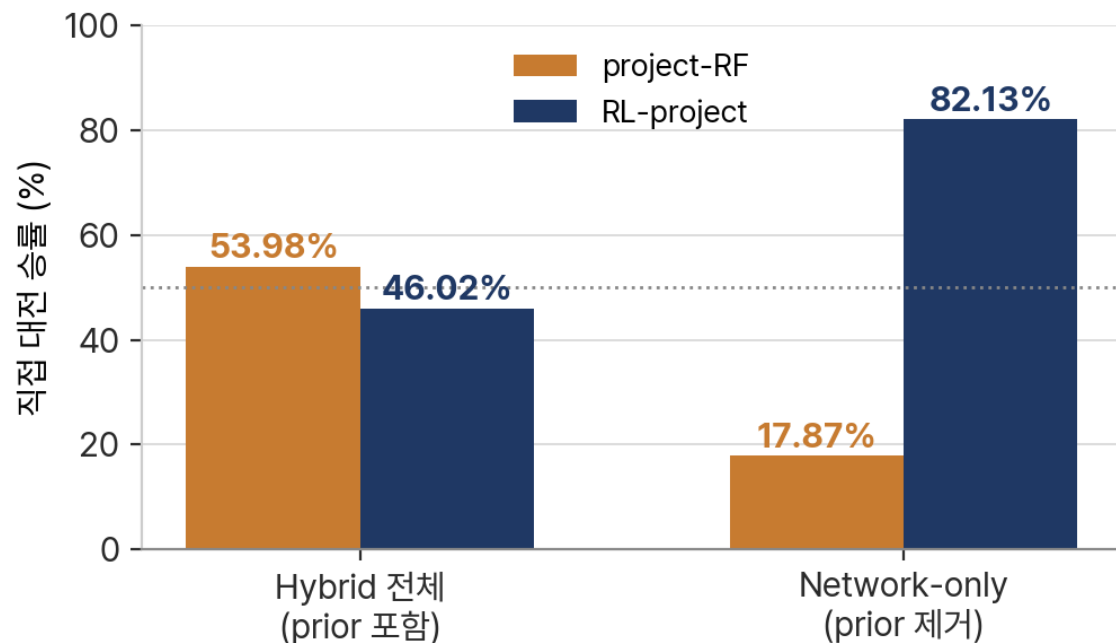
RL 40M seed 1 vs project-RF capture imitation (paired 5,000판)

해석

- project-RF의 게임당 잡기 수가 정확히 약 2배 (4.49 vs 2.21)
- tactical prior가 잡기 기회와 상대의 즉시 반격을 직접 평가한 결과
- RL 정책끼리의 대전에서는 이 공격적 잡기가 상대 진행을 계속 리셋
- 고정 baseline 상대 승률에는 잘 드러나지 않던 matchup 강점

Ablation — Tactical Prior를 제거하면

같은 project-RF Checkpoint에서 Prior만 OFF 후 동일 조건 재대전



직접 대전 15,000판 — prior 유무에 따른 역전

-36.11%p

Prior 제거 시 project-RF 승률 변화
(53.98% → 17.87%)

대전 결과

- project-RF network가 더 강하다”는 해석과 달랐다.
신경망 단독 비교에서는 RL-project가 82.13%로 우세
- project-RF의 직접 대전 우위는 network 자체가 아니라, rule-based prior가 결합된 hybrid 추론 구조에서 발생
- 즉, **project-RF Hybrid agent가 해당 Matchup에서 우세했다고 해석하는 것이 타당하다고 생각함**

평가 기준에 따라 순위가 달라지는 이유

- 1 RL 성능은 상대의 함수** 강화학습 agent의 승률은 절대값이 아니라 어떤 상대를 만나는가에 따라 달라진다
- 2 RL-project = baseline 특화** 고정 common rule의 deterministic 분포에 직접 40M 학습 → baseline 상대 최고 성능 성능
- 3 project-RF = matchup 대응** mixed opponent 학습 + 상대의 즉시 반격을 평가하는 prior → 낫선 정책에도 강함
- 4 prior를 빼면 역전** network 단독 품질은 장기 학습한 RL-project가 압도 (82.13%)

결론: 하나의 승률만으로는 어떤 agent가 더 강하다고 단정하기 어렵기 때문에, 여러 평가 조건에서의 결과를 함께 봐야 한다.

설계 방식에 따른 성능 차이

Project_RF

<설계 방식>
지식을 정책 판단 단계에 결합

<강점>

1. 적은 학습량으로 전술적 행동 수행 가능
2. 행동 선택 근거 설명 가능
3. 직접 대전 matchup 대응력 우세

RL-project

<설계 방식>
지식을 관측 입력 단계에 결합

<강점>

1. Network 단독으로 정책 수행가능
2. 공통 baseline 대비 목표 수준 달성
3. 반복 실험에서 비교적 안정적 결과

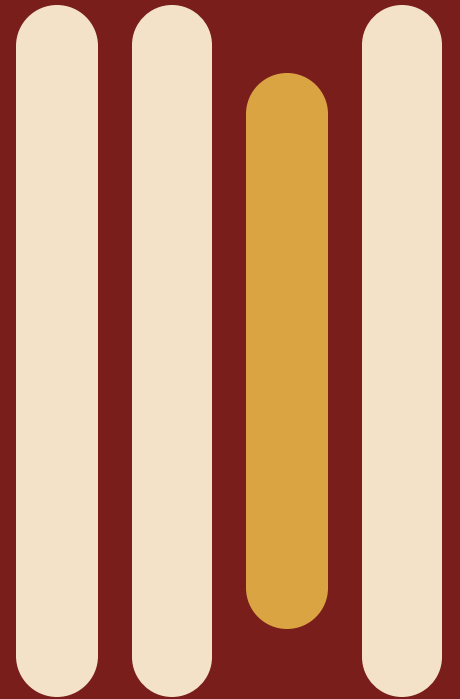
결론

- 두 접근 모두 공통 baseline 대비 약 60% 수준에 도달함
- 성능 차이는 모델 우열보다 지식 주입 위치의 차이에서 발생
- 지식을 정책 판단에 넣으면 즉시성과 설명 가능성이 높아짐
- 지식을 관측에 넣으면 network 중심의 일반화 가능성이 커짐

어느 모델이 더 강한가 보다 어떤 방식으로 지식을 활용했는가에서 차이를 보임

04

해석과 결론



공정성과 한계

이 비교는 “우열 판정”이 아니라 “두 설계 사례의 비교”다

학습 조건의 비대칭

- RL-project는 3개 seed 평균, project-RF는 단일 checkpoint 기준으로 비교됨
- 학습 budget과 인프라 조건이 달라 sample efficiency 직접 비교는 어려움
- RL-project observation에도 rf_score가 포함되어 있어, 양쪽 모두 rule 기반 지식을 일부 활용함

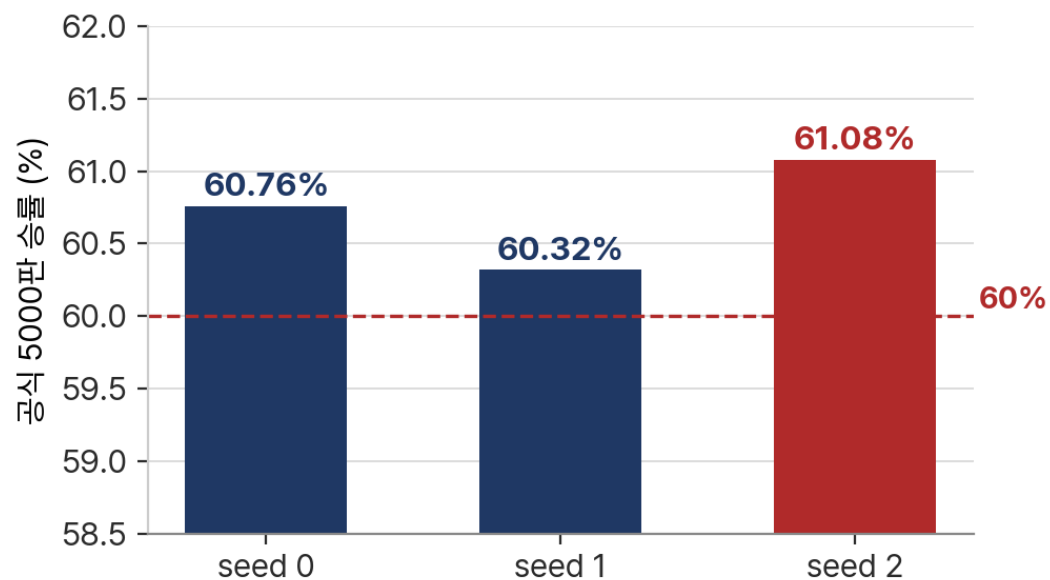
환경 및 평가 조건의 차이

- A3·A4 위치는 거리 기반 투영 방식으로 변환되어 일부 domain shift 가능성이 있음
- 직접 대전은 뒷도가 없는 legacy 규칙 부분집합에서 수행됨
- 일부 직접 대전은 기존 공통 평가 seed를 재사용했으며, 이를 보완하기 위해 holdout 평가를 추가 수행함

재현성의 한계

- 일부 실험 당시 코드·설정 버전과 최종 repository 상태 간 일치 여부 확인 필요
- 모델 파일뿐 아니라 config, ruleset, 코드 버전을 함께 보존해야 재현 가능
- 최종결과 중 51.32%는 단일 평가 결과와 53.98%는 3-seed 기준 결과이므로 동일한 수치로 해석하면 안 됨

RL-project 의 후속 mini 프로젝트 — Full-backdo 50M



뒷도 포함 환경 공식 paired 5,000판 — 3개 seed 모두 60% 통과

추가 내용

- 후속 확장 실험
뒷도 포함 full-backdo 환경에서 seed당 50M step으로 재학습
- 성능 향상 확인
기존 대비 승률이 상승하여 장기 학습의 효과를 확인

해석

충분한 학습 시간과 더 완전한 규칙 환경이 제공된다면, RL-project의 추가 성능 향상을 기대할 수 있음

핵심 결론

- 1** 두 설계 모두 목표 수준 달성 공통 baseline 대비 project-RF 59.76%, RL-project 59.46% (목표 60%)
- 2** 직접 대전은 project-RF 강함 holdout 15,000판 53.98%, 잡기 중심 prior의 matchup 대응력
- 3** 신경망 단독은 RL-project 강함 network-only 대결 82.13% — 장기 학습 + tactical observation이 만든 정책 품질
- 4** 우위는 평가 기준에 따라 달라짐 baseline 성능, matchup 대응력, 정책 품질 이 서로 다른 결과를 보임
- 5** 설계 방식의 차이 확인 RL-project: 관측 입력에 지식 반영
project-RF: 출력 단계에 rule-based prior 결합

향후 방향

비교 완성도 개선

- 동일한 ruleset, action space, seed, 학습 budget 기준에서 재평가하여 비교의 공정성을 높임

각 모델 개선

- RL-project는 장기 학습과 observation/reward 고도화, project-RF는 hybrid prior와 network policy의 결합 방식 개선

환경 통합

- 뒤도 포함 여부, action 정의, board 표현을 통합하여 두 agent가 동일 환경에서 직접 비교될 수 있도록 정리

팀원별 기여 사항

남준우 — RL-project

- **공통 기반 구축**
윗놀이 규칙 엔진, Gymnasium 환경, 자동 테스트 체계 구현
- **RL-project 설계 및 학습**
Action-level tactical observation, terminal reward 설계
MaskablePPO 3M~40M 학습 및 ablation 실험 수행
- **공통 평가 체계 구축**
평가 프로토콜 및 cross-env adapter 구현
동일 조건 기반 agent 비교 가능 구조 설계
- **확장 실험 수행**
full-backdo 환경 확장 및 50M step 장기 학습 실험 진행

이기쁨 — project-RF

- **환경 및 agent 구현**
윗놀이 환경과 Rule, DQN, Value, StrategicValue agent 구현
- **상태·보상 설계**
252차원 engineered state와 capture-aware dense reward 설계
- **학습 전략 구성**
StrategicValue teacher imitation, oversampling, 자체 PyTorch PPO 적용
- **Hybrid 추론 구조 설계**
inference-time tactical prior를 결합한 의사결정 구조 구현
- **평가 수행**
내부 tournament 및 RL-project와의 직접 대전 평가 진행

공동 게임 규칙·action 의미·평가 조건 합의, 공통 Rule-based Agent 정의, 교차 평가 결과 해석과 보고서 작성

감사합니다

